

Proposed resolution for US70-126: allow incomplete types in type_order

Reply-to: Gašper Ažman gasper.azman@gmail.com

Date: 2026-03-23

Paper number: P4140R0

Context

The `type_ordering` metafunction (17.11.7 [compare.type](#)) is specified in terms of a literal struct definition

```
template<class T, class U> struct type_order {  
    static constexpr strong_ordering value = TYPE-ORDER(T, U);  
  
    using value_type = strong_ordering;  
  
    constexpr operator value_type() const noexcept { return value; }  
    constexpr value_type operator()() const noexcept { return value; }  
};
```

It used to be defined in terms of [Cpp17BinaryTypeTrait](#), but changed this in [P3778R0](#) due to `std::strong_ordering` not being a structural type.

During that change, the explicit allowance for `type_order<X, Y>` being defined for incomplete `X` or `Y` got inadvertently dropped, so this paper re-introduces that allowance.

This is not a change in design - this is a fixup to make the wording consistent with the design as passed in [P2830R10](#).

Proposed Wording

2. If an explicit specialization or partial specialization of `type_order` is declared, the program is ill-formed.
3. The templates `type_order` and `type_order_v` may be instantiated with incomplete types as arguments.
4. Recommended practice: The order should be lexicographical on parameter-type-lists and template argument lists.